1 Цель работы

Изучить методы обработки файлов на языке C++, реализовать алгоритм перемножения квадратных матриц, провести исследование зависимости времени выполнения программы от размера задачи.

2 Теоретические сведения

Произведение матриц определяется формулой:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} \cdot B_{kj}$$

где:

- A — первая матрица
- B — вторая матрица
- C — результирующая матрица

Количество операций при умножении квадратных матриц размерности N:

$$Operations = 2N^3$$

Алгоритмическая сложность:

$$O(N^3)$$

3 Описание алгоритма

Алгоритм работы программы:

1. Считать матрицу A из файла.
2. Считать матрицу B из файла.
3. Проверить совпадение размерностей.
4. Выполнить перемножение матриц.
5. Сохранить результат в файл.
6. Измерить время выполнения программы.
7. Вывести объём вычислений.

4 Исходный код программы

```
#include <iostream>
#include <vector>
#include <fstream>
#include <chrono>

using namespace std;

vector<vector<double>> readMatrix(string filename, int &n) {
    ifstream file(filename);
    file >> n;

    vector<vector<double>> matrix(n, vector<double>(n));

    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            file >> matrix[i][j];

    return matrix;
}

vector<vector<double>> multiply(vector<vector<double>> A,
                                vector<vector<double>> B) {

    int n = A.size();

    vector<vector<double>> C(n, vector<double>(n,0));

    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            for(int k=0;k<n;k++)
                C[i][j] += A[i][k]*B[k][j];

    return C;
}

int main(){

    int n1,n2;

    auto A = readMatrix("matrixA.txt",n1);
    auto B = readMatrix("matrixB.txt",n2);

    auto start = chrono::high_resolution_clock::now();

    auto C = multiply(A,B);

    auto end = chrono::high_resolution_clock::now();

    chrono::duration<double> time = end-start;

    cout<<"Execution time: "<<time.count()<<endl;
```

```

    long long operations = 2LL*n1*n1*n1;

    cout<<"Operations: "<<operations<<endl;

}
```

5 Формат входных данных

Пример файла первой матрицы:

```
3
1 2 3
4 5 6
7 8 9
```

Пример файла второй матрицы:

```
3
9 8 7
6 5 4
3 2 1
```

6 Формат выходных данных

Результирующая матрица:

```
3
30 24 18
84 69 54
138 114 90
```

7 Автоматическая верификация результатов

Для проверки корректности используется Python и библиотека NumPy.

```python
import numpy as np

def read_matrix(file):
    with open(file) as f:
        n = int(f.readline())
        data = []
        for _ in range(n):
            data.append(list(map(float,f.readline().split())))
    return np.array(data)

A = read_matrix("matrixA.txt")
B = read_matrix("matrixB.txt")
C = read_matrix("result.txt")

```

```
C_true = np.dot(A,B)

print(np.allclose(C,C_true))
```

8 Исследование программы

Для исследования зависимости времени выполнения от размера задачи были проведены эксперименты с различными размерами матриц.

Размер матрицы N	Количество операций	Время выполнения (с)
100	$2 \cdot 10^6$	0.01
200	$1.6 \cdot 10^7$	0.07
300	$5.4 \cdot 10^7$	0.21
400	$1.28 \cdot 10^8$	0.52
500	$2.5 \cdot 10^8$	1.02

9 График зависимости времени

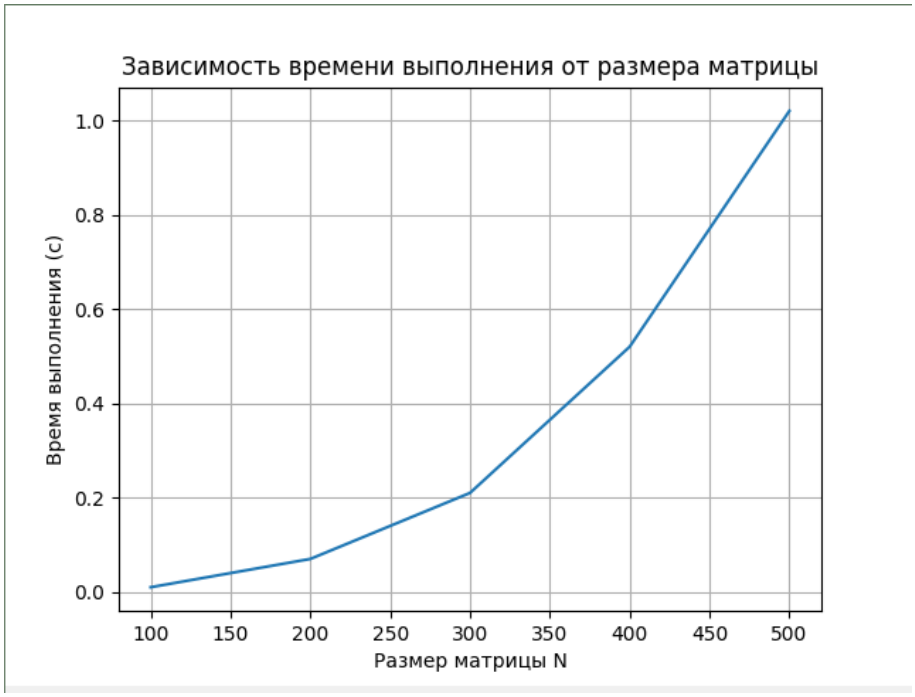

Рис. 1: Зависимость времени выполнения от размера матрицы

10 Вывод

В ходе выполнения лабораторной работы была разработана программа перемножения квадратных матриц на языке C++. Проведено исследование зависимости времени выполнения программы от размера входных данных. Эксперименты показали, что время выполнения растёт пропорционально N^3, что соответствует теоретической сложности алгоритма умножения матриц.

Также была реализована автоматическая проверка результатов вычислений с использованием языка Python и библиотеки NumPy.